Solar System Simulator — Desktop App

A Tkinter desktop UI for the solar/battery sound system rule engine.
No installation required — uses Python's built-in `tkinter` library only.

Launch

```
cd "C:\Users\AlexandraCabral\Documents\Bike"
.\.venv\Scripts\Activate.ps1
python app.py
```

The window opens pre-filled with your system from the diagram.

Layout

```
┌─────────────────┬─────────────────┬─────────────────────┐
│ ☀ Solar Panel   │ 🔋 Battery      │ ⚡ Loads             │
│ ⚙ Controller    │ 🔌 Inverter     │ 🌥 Environment       │
│                 │                 │ 📋 Policy           │
├─────────────────┴─────────────────┴─────────────────────┤
│              ▶ Simulate  (full-width button)            │
│              Output / Results text box                  │
└─────────────────────────────────────────────────────────┘
```

Column 1 — Solar + Controller

Panel	What you can edit
☀ Solar Panel	Rated power (W), Voc, Vmp, Isc, Imp, temp coefficient β, series count, parallel count
⚙ Controller	Type (MPPT/PWM), max PV Voc, max charge amps, supported battery voltages (comma-separated, e.g. `12,24`), PV power limit at 12 V and 24 V, Vmp margin

Column 2 — Battery + Inverter

Panel	What you can edit
🔋 Battery	Chemistry, nominal voltage, capacity (Ah), max depth of discharge, BMS continuous current, BMS peak current
🔌 Inverter	DC input voltage, continuous power, surge power, efficiency, idle draw, pure sine checkbox

Column 3 — Loads + Environment + Policy

Panel	What you can edit
⚡ Loads	5 fixed loads (one box each): power (W), hours/day, type (AC/DC), surge (W), required DC voltage
🌥 Environment	Peak sun hours, min temperature (°C), PV derate factor
📋 Policy	Autonomy hours, energy margin, controller headroom, fuse factor

---

Running a simulation

1. **Edit any field** you want to test — for example, change battery capacity, panel count, or inverter size.
2. Click ▶ **Simulate**.
3. Read the output box:

Output — PASS

```
──────────────────────────────────────────────────────
  ✅ PASS      Bus Voltage: 24 V
──────────────────────────────────────────────────────
  ENERGY BUDGET
    DC loads              :    1600.0 Wh
    ...
  BATTERY
    Min Ah needed :   147.3 Ah   installed: 150.0 Ah
    ...
  All checks passed. Configuration is electrically valid.
```

Output — FAIL

```
──────────────────────────────────────────────────────
  ❌ FAIL      Bus Voltage: 24 V
──────────────────────────────────────────────────────
  ...
  1 FAILURE(S) — fix before purchasing
  [1] Battery too small: need 402 Ah, have 50 Ah.

      WHY: The battery does not hold enough energy ...
      FIX: Use a battery with at least 402 Ah at 24 V ...
```

Each failure shows:

- A **one-line summary** of what failed
- **WHY** — the electrical risk if you ignore it
- **FIX** — a concrete corrective action

Common simulations to try

What to test	How
Battery too small	Set `Capacity (Ah)` to `50`, click Simulate
Wrong inverter voltage	Set inverter `DC input (V)` to `12`, click Simulate
Too few solar panels	Set `Parallel count` to `1`, `Series count` to `1`, click Simulate
Controller overloaded	Set `Max charge A` to `10`, click Simulate
12 V system	Set `Battery → Voltage nom (V)` to `12`, inverter `DC input (V)` to `12`, click Simulate
Modified-sine warning	Uncheck `Pure sine` on the Inverter panel, click Simulate

---

Files used by the app

File	Role
`app.py`	This UI — the only file you run
`rules_engine.py`	All electrical rules (do not edit)
`simulate.py`	CLI version + `builtin_example()` default values
`my_system.json`	Reference config (not loaded by the UI — edit fields directly)